

Social Network

Conventions & Liste de Tâches

1. Nomenclature & Conventions

Langue

- Commentaires, branches et commits → **français**
- Noms de variables, fonctions, fichiers → **anglais**

Format des noms

Format	Usage
camelCase	Variables et fonctions
PremièreLettreMaj	Structs, Classes, Objets
ID	Le mot « id » — toujours écrit en majuscules
snake_case	JSON : tout en minuscules avec underscores
snake_case	BDD : tout en minuscules avec underscores

Commentaires JS — JSDoc

Les commentaires de fonctions utilisent la syntaxe **JSDoc** : taper `/**` puis *Entrée* pour générer automatiquement le bloc. Cela active l'autocomplétion et la documentation inline dans l'IDE.

```
/**
 * Calcule le score d'un utilisateur.
 * @param {string} userID - L'identifiant de l'utilisateur.
 * @returns {number} Le score calculé.
 */
function calculateScore(userID) { ... }
```

Branches Git — format type/nom

Format : `type/nom-court-en-français-avec-tirets`

```
feature/creation-de-post
fix/validation-formulaire-inscription
chore/configuration-docker
```

Préfixe	Usage
feature/	Nouvelle fonctionnalité visible par l'utilisateur
fix/	Correction d'un bug
refactor/	Réécriture sans changer le comportement
docs/	Documentation uniquement (README, commentaires)
test/	Ajout ou modification de tests
perf/	Amélioration de performance sans changement de comportement
style/	Mise en forme, indentation, ponctuation — sans impact sur le code
build/	Modifications du système de build ou des dépendances (Docker, packages)
ci/	Configuration des pipelines CI/CD (GitHub Actions, etc.)
chore/	Tâches techniques sans impact visible (scaffolding, config)
revert/	Annulation d'un commit précédent

Commits — format type(scope): description

Format officiel : `type(scope): description`

```
type(scope): description courte en français
```

```
Corps optionnel : explication plus détaillée si nécessaire.
```

```
Footer optionnel : Refs #42, BREAKING CHANGE: ...
```

Règles :

- Description courte, en français, sans majuscule en début, sans point final.
- Le scope (entre parenthèses) est optionnel mais recommandé : auth, groupe, post, chat, bdd, notif...
- Un **!** après le type signale un **breaking change** : une modification qui casse la compatibilité avec ce qui existait avant — par exemple changer le format d'une réponse API, renommer un champ en BDD, ou supprimer une route. Tout code qui dépendait de l'ancien comportement devra être mis à jour. Le détail doit être explicité dans le footer avec **BREAKING CHANGE** :

Exemples :

```
fix(post): corriger l'affichage des likes sur mobile
perf(bdd): optimiser la requête de récupération des posts
```

Type	Usage
feat	Nouvelle fonctionnalité
fix	Correction de bug
refactor	Réécriture sans changer le comportement (ni fix, ni feat)
docs	Documentation, commentaires, README
test	Ajout ou modification de tests
perf	Amélioration de performance
style	Mise en forme pure (espaces, indentation) sans impact logique
build	Build, dépendances, configuration des packages
ci	Pipelines CI/CD
chore	Tâches techniques sans impact visible (scaffolding, outillage)
revert	Annulation d'un commit précédent

2. Liste de Tâches

Infrastructure & Configuration

- Initialiser les deux conteneurs Docker (frontend / backend)
- Configurer PostgreSQL + GORM
- Créer les migrations et le jeu de données de démonstration
- Mettre en place le serveur, les middlewares et les routes
- Rédiger la procédure de travail (soumission de PR, étapes de validation)
- Mettre en place un blocage de connexion après trop de tentatives échouées
- Tester bun vs pnpm comme gestionnaire de paquets

Authentification

- Inscription (validation des champs, unicité email/pseudo, gestion des homonymes)
- Connexion (cookie de session, maintien de connexion)
- Déconnexion (suppression du cookie)

Pages Frontend

- Accueil (feed groupes actifs, posts publics, bouton inscription)
- Connexion & Inscription
- Hub, Profil, Section Utilisateur
- Post, Liste des Groupes, Groupe
- Chat(s), Notifications

En-tête (toutes les pages)

- Logo/symbole projet (redirection selon état de connexion)
- Barre de recherche (tag, créateur, nom de post/groupe)
- Bouton Déconnexion (visible si connecté)
- Bouton Section Utilisateur
- Bouton Liste des Groupes
- Bouton Création de Post (pop-up ou page dédiée — à trancher en équipe)
- Bouton Hub
- Onglet notifications (badge + liste 3 dernières + bouton page notifications)

Posts & Commentaires

- Création/suppression de post (avec image JPEG/PNG/GIF, max 5 Mo)
- Règles d'accessibilité (public, restreint, privé)
- Commentaires, Likes/Dislikes sur posts et commentaires
- Redirection automatique vers le feed après création d'un post

Groupes

- Création / suppression de groupe
- Transmission des droits créateur
- Demande d'accès, acceptation/refus (avec historique candidat affiché au créateur)
- Bannissement (avec raison optionnelle stockée en BDD)
- Posts et commentaires internes au groupe (sans règle d'accessibilité)
- Événements de groupe (titre, description, date, options inscription/refus)
- Chat de groupe (WebSocket)

Chat Privé

- Chat entre deux utilisateurs (WebSocket)
- Notification temps réel à la réception d'un message
- Support des emojis
- Tri de la liste : alphabétique sans historique, par message le plus récent sinon

Liste des Personnes Connectées/Déconnectées

- Accessible sur toutes les pages (à confirmer en équipe)
- Codes couleur : bleu (suivi), vert (follower), rouge (mutuel)

- Actions selon relation : Suivre / Envoyer un message / Voir le profil

Notifications

- Demande de suivi (profil privé)
- Invitation à un groupe
- Demande d'accès à un groupe (vue créateur)
- Nouveau post dans un groupe où l'utilisateur est inscrit
- Nouveau commentaire sur un post de l'utilisateur
- Nouveau commentaire sur un post où l'utilisateur a participé
- Nouvel événement dans un groupe
- Refus d'accès à un groupe

Qualité & Tests

- Tests unitaires JS avec Jest
- Tests unitaires Go (natif)
- Rédiger les normes JS et Go dans un document dédié
- Explorer le stockage de JSON en BDD pour les données flexibles (notifications, bannissements...)